

Randers-UMAP on the Swiss Roll: Three Initialisation Strategies and the Ramp Mechanism

Progress Report, Week of August 7–12

August 12, 2026

1 Methodology recap

All three pipelines below share the same skeleton.

Distance matrix. `randers_bridge.py::compute_dist_matrix` turns (X, ω) into an asymmetric geodesic distance matrix D_{asym} : build a neighbourhood graph on X , perturb each edge weight by $\langle \omega_i, x_j - x_i \rangle$, run directed shortest-path. This week the neighbourhood-graph step was rewritten (§4) to match [lwileczek/isomap](#)’s threshold construction instead of scikit-learn’s k -NN graph.

Randers metric. Every method ultimately positions each node i at $y_i \in \mathbb{R}^2$ together with a drift vector b_i , $\|b_i\| \leq 1 - \delta$, so that

$$F(i \rightarrow j) = \|y_j - y_i\| + \langle b_i, \hat{e}_{ij} \rangle \quad (1)$$

approximates $D_{\text{asym}}[i, j]$. The three pipelines differ only in *how Y is initialised* and *what update rule* then refines Y – see §3.

Force-directed update `randers_umap.py::randers_umap_fit` lays Y out with a dense, full-batch UMAP-style attraction/repulsion graph layout, not gradient descent against a loss. D_{asym} is first turned into a fuzzy membership graph $\mu_{ij} \in [0, 1]$ exactly as in UMAP itself (Section 3.1 of the original paper): a per-point ρ_i, σ_i calibration on the k nearest neighbours so every node’s total soft-neighbourhood weight matches $\log_2 k$. The drift then enters the layout by replacing the ordinary embedding distance and force direction with

$$\rho_{i \rightarrow j} = \|y_j - y_i\| + b_i \cdot (y_j - y_i), \quad g_{ij} = \hat{e}_{ij} + b_i, \quad (2)$$

inside UMAP’s own attractive/repulsive coefficients $\text{attr}(\rho), \text{rep}(\rho)$ (the usual (a, b) curve fit to `min_dist/spread`). The net force on node i is

$$\text{force}_i = \sum_j \left[\mu_{ij} \text{attr}(\rho_{ij}) - (1 - \mu_{ij}) \text{rep}(\rho_{ij}) \right] g_{ij}, \quad (3)$$

and Y is updated by pure force accumulation, $Y \leftarrow Y + \eta \text{force}$, every epoch. Setting $b_i \equiv 0$ everywhere recovers vanilla UMAP exactly, since then $\rho = \|y_j - y_i\|$ and $g = \hat{e}_{ij}$.

Locating the drift via virtual points. B itself is not learned by this force loop – `B_fixed` is computed once, *before* training, and never updated afterwards (only its *magnitude* is phased in, see the ramp mechanism, §2). The main construction locates it geometrically: for every real point x_i , place an imaginary target

$$x_i^{\text{virt}} := x_i + \omega_i \quad (4)$$

in the same ambient space – a genuine position, not an abstract offset, since ω is represented in the same coordinates as X . All $2n$ real+virtual points are then embedded *jointly* with plain, drift-off UMAP ($b \equiv 0$ in Eq. (2)) on their symmetric geodesic distance. The located drift is simply the displacement between the two embedded images of the same node,

$$B_{\text{located}} := y_i^{\text{virt}} - y_i^{\text{real}}, \quad b_i \leftarrow b_i \cdot \min\left(1, \frac{1-\delta}{\|b_i\|}\right), \quad (5)$$

clipped to the Randers bound. B_{located} is then frozen and passed as **B.fixed** to the real (asymmetric) embedding: it still shapes Y 's force every epoch through Eq. (2), but its own value never changes for the rest of training – it is rigidly “attached” to node i and rides along as y_i moves.

2 The ramp mechanism

2.1 What it is

`randers_umap_fit` accepts a frozen drift field **B.fixed** (computed once, attached to each node, never retrained – see §3). A scalar schedule $s(\tau)$ optionally controls how much of that frozen field is actually applied at epoch τ :

$$s(\tau) = \begin{cases} 0 & \tau/T \leq 0.30 \\ (\tau/T - 0.30)/0.40 & 0.30 < \tau/T < 0.70, \\ 1 & \tau/T \geq 0.70 \end{cases}, \quad B(\tau) = s(\tau) \cdot B_{\text{fixed}}, \quad (6)$$

T the total epoch count. **ramp=False** fixes $s \equiv 1$: the frozen field is applied at full strength from epoch 0. Only s changes with τ – B_{fixed} 's direction is never touched, so this is not a re-optimisation of B , only a magnitude schedule.

2.2 Why full strength from epoch 0 can fail

From (1), Cauchy–Schwarz bounds the drift term:

$$|b_i \cdot (y_j - y_i)| \leq \|b_i\| \cdot \|y_j - y_i\|. \quad (7)$$

So $\rho_{i \rightarrow j} \in [(1 - \|b_i\|)d_{ij}, (1 + \|b_i\|)d_{ij}]$. For a well-behaved node ($\|b_i\| \approx 0.05$, the typical case at $n = 300$, see Table 1) this is a gentle $\pm 5\%$ perturbation of distance. For a node sitting at the Randers clip boundary ($\|b_i\| \approx 0.99$) it is a $\pm 99\%$ swing: ρ can fall to $\approx 0.01 d_{ij}$ or rise to $\approx 1.99 d_{ij}$ for the *same* pair of points, depending only on which node is source. The (a, b) attraction/repulsion curve was calibrated (**min_dist=0.1**, **spread=1.0**) assuming ρ behaves like an ordinary, well-scaled UMAP distance – not one that can swing this violently. Applying that swing at full strength ($s = 1$) from epoch 0, while Y is still the compact, just-finished output of the locate step and has not yet found its natural scale, produces erratic per-edge forces for every saturated node: a minority of nodes get flung to extreme positions while the majority, left with only ordinary (and, at this n , comparatively weak dense-repulsion) attraction, collapse toward a shared centroid.

2.3 Empirical evidence, $n = 2000$

With **ramp=False** on the 206/2000-saturated run: apply-step epoch 0 already shows mean $|\rho - d| = 0.797$ (against 0.124 at $n = 300$, 0 saturated nodes), and the final embedding extent reaches 149.6 (against 19–24 at $n = 300$) – see Figure 1a: a tight central clump with a handful of nodes flung to the edges. Switching only **ramp=True**, same data, same seed, same B_{located} : Figure 1b shows a clearly organised, colour-separated layout by epoch 20–40, because $s \approx 0$ during the first 30% of epochs lets Y spread under ordinary UMAP forces *before* the saturated nodes' extreme drift is switched on.

run	mean $\ b\ $	max $\ b\ $	clipped (of 2000)
$k = 20$	0.147	0.990	217
$k = 12$	0.145	0.990	218

Table 1: differnt run results for k=12 and k=20

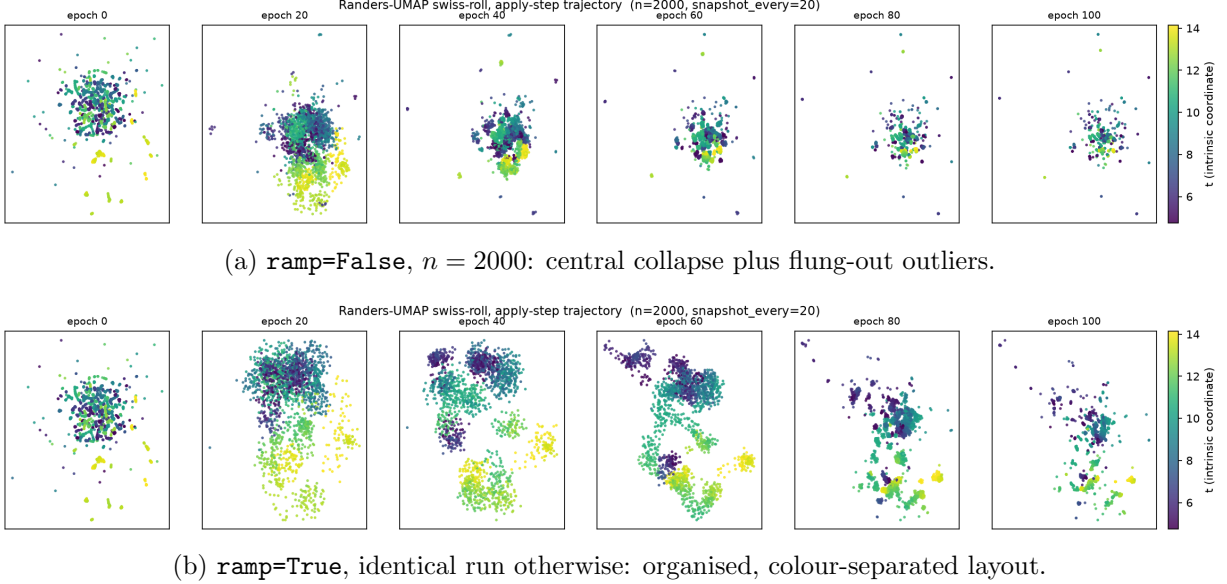


Figure 1: Effect of the ramp schedule on the same B_{located} , $n = 2000$.

2.4 Caveat – not yet fully isolated

Table 1’s two runs differ in *both* k (20 vs. 12) *and* locate-step epoch count (500 vs. 100) simultaneously, because the short-locate-epoch run was originally set up for fast iteration, not as a controlled comparison. The saturation jump is more plausibly explained by the shorter, under-converged locate step than by k itself (see §4’s k discussion, which is a separate, independently-verified finding). A clean single-variable rerun ($k = 12$, locate-epochs=500) is discussed further in the analysis below (§5) but has not been run yet. **ramp=True** was adopted regardless because it makes the pipeline robust to a saturated B_{located} *whatever* its source, which is a reasonable thing to want independent of this open question. **ramp** is now a CLI flag (`--ramp`, default off) on all three scripts in §3, so it can be toggled without editing code.

3 Three initialisation strategies

All three now share the same force-directed update mechanism, Eq. (2). They differ only in how Y_{init} is obtained.

3.1 Isomap – `run_swiss_roll.py`

D_{asym} from `compute_dist_matrix` (the Isomap-style geodesic backbone). B is *located*: place a virtual target $x_i^{\text{virt}} := x_i + \omega_i$ for every real point, embed all $2n$ real+virtual points jointly with plain (drift-off) UMAP on their symmetric geodesic distance, then $B_{\text{located}} := y_i^{\text{virt}} - y_i^{\text{real}}$. Y_{init} is simply y^{real} from that same joint embedding. Figure 2.

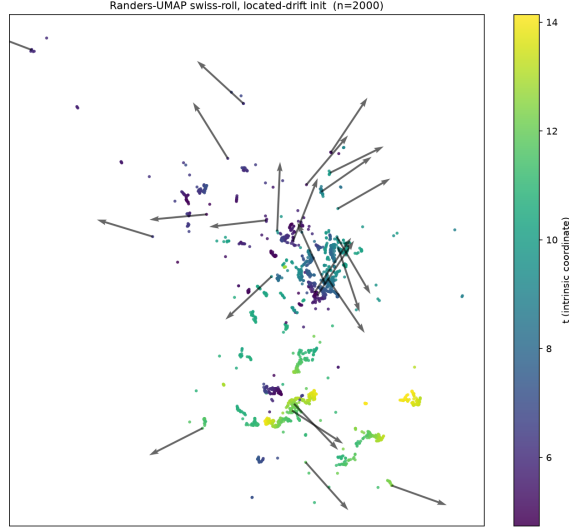


Figure 2: $n=2000$, 100 epochs

3.2 isumap – `run_swiss_roll_isumap.py`

D_{asym} comes from `distance_graph_generation.py` – isumap’s own, untouched library code (local ρ_i/σ_i normalisation, star-graph restriction, fuzzy t -conorm merge – see the file-by-file walkthrough discussed separately). This method has no notion of ω at all – its asymmetry comes purely from the directed k -NN structure isumap itself builds – so there is no virtual target to locate. B instead comes from a one-shot SVD of the skew-symmetric part, $B := \text{svd_init}(D_{\text{asym}}^\top - D_{\text{asym}})[:, : d]$, clipped to the Randers bound. Y_{init} is whatever `randers_umap_fit`’s own spectral initialisation gives on D_{asym} (no separate locate step here either). Figure 3.

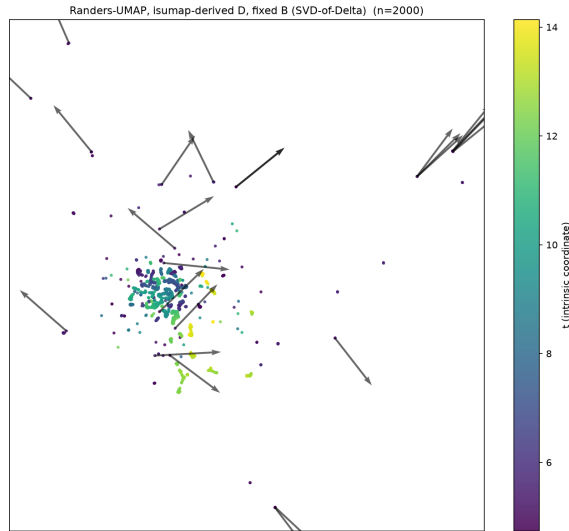


Figure 3: $n=2000$, 100 epochs

3.3 SVD-RADAR – `run_swiss_roll_svd_radar.py`

Same D_{asym} as the Isomap pipeline (true ω). B is *still* the located construction above – unchanged, because B needs the true ω field to locate against, and SVD-of-Delta is a structurally different signal not yet validated as a substitute. The only thing that changes relative to §3.1 is Y_{init} : instead of the locate step’s own y^{real} , $Y_{\text{init}} := \text{svd_init}(D_{\text{asym}})[:, : d]$ (RADAR-style truncated

SVD of the asymmetric matrix itself, no double-centering). This isolates a single variable against §3.1: does an SVD-derived starting position help or hurt, everything else held fixed. (This script previously used a completely different mechanism – SVD init feeding an Adam/Finsler-loss regression, `finsler_mds_freeB` – see §4.)

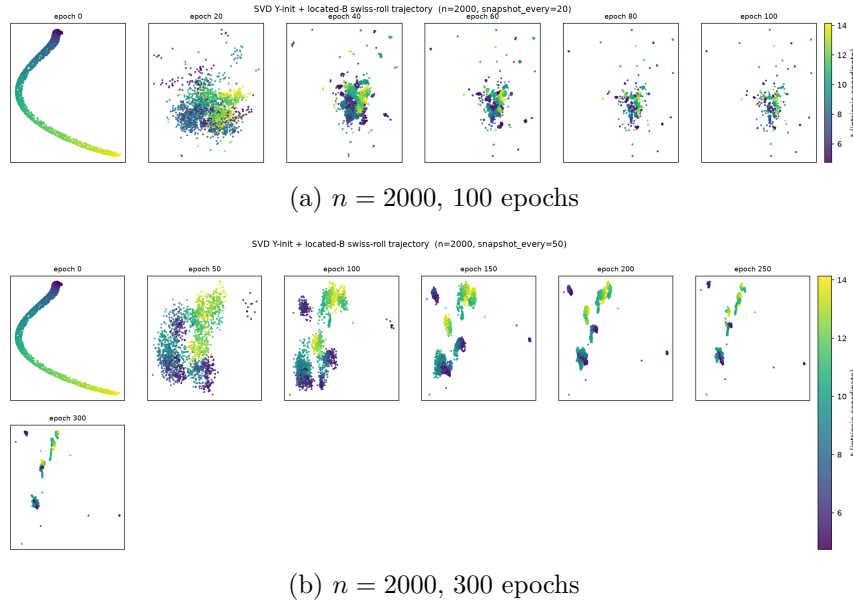


Figure 4: SVD-RADAR result at 100 vs. 300 epochs.

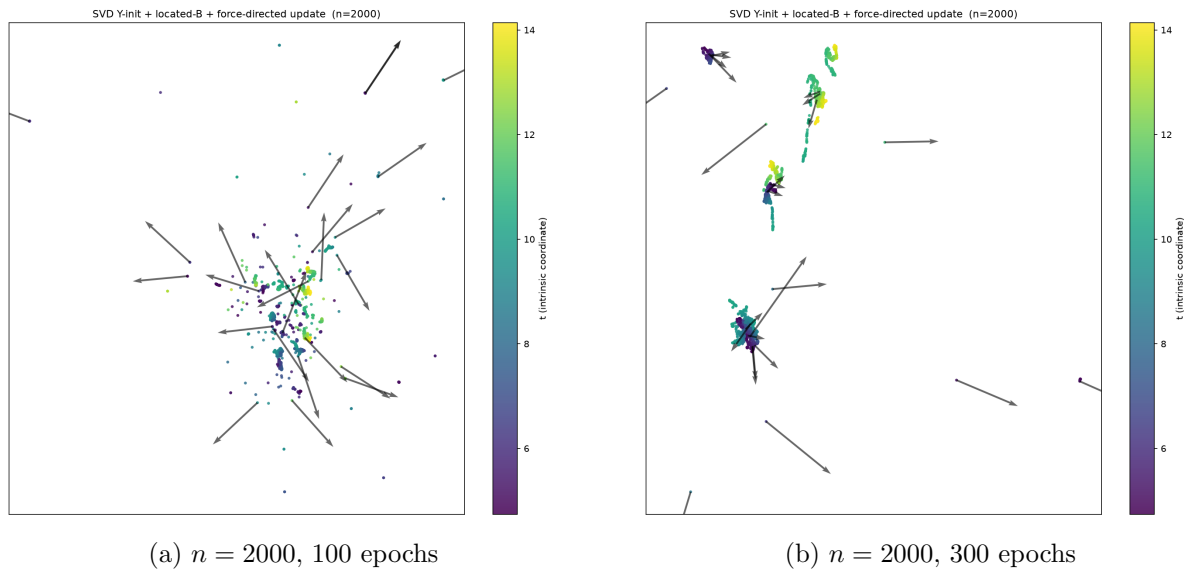


Figure 5: SVD-RADAR final embeddings at 100 vs. 300 epochs.

4 This week's changes

1. **Geodesic backbone rewritten to match `lwileczek/isomap`.** `compute_dist_matrix` previously built its neighbourhood graph via `scikit-learn's kneighbors_graph` (exactly k neighbours per point, adaptive to local density). It now uses a single global distance threshold ε ($\text{dist} < \varepsilon \Rightarrow \text{edge}$), matching `lwileczek/isomap's make_adjacency` at the advisor's request. ε is auto-derived from k so existing call sites are unaffected: $\varepsilon(k) = \max_i d_k(i)$, the largest per-point k -th-nearest-neighbour distance in the dataset.

2. **Diagnosed and fixed geodesic-backbone contamination.** Because $\varepsilon(k)$ is a single *global* value (set by the sparsest point), points in denser regions can end up connected to ambient neighbours across different windings of the roll – a classic Isomap short-circuit. Measured directly: at $n = 300$, $k = 20$, the true geodesic 20-NN of some points reach as far as $|t_i - t_j| = 6.87$ (75% of the entire t -range) – structurally wrong. Reducing k (20→12) is provably safe against this specific failure mode, since ε is monotone non-decreasing in k : at $n = 2000$, $k = 12$, the same statistic drops to max 0.58, median 0.11 – clean. (This is independent of the ramp finding in §2, which turned out to hinge more on locate-step convergence than on k – see the caveat there.)
3. **snapshot_every added to both update mechanisms.** Both `randers_umap_fit` and `finsler_mds_freeB` now record (Y, B) every N epochs (epoch 0 = true pre-training init, final epoch always included even under early stopping), enabling the trajectory grids used throughout this report and Figures 1a–1b.
4. **ramp exposed as a CLI flag everywhere.** `run_swiss_roll.py`, `run_swiss_roll_isomap.py`, and `run_swiss_roll_svd_radar.py` all take `--ramp` now (default off, matching the "locate once, freeze, attach at full strength" design intent) – previously `run_swiss_roll_isomap.py` silently used `randers_umap_fit`'s own `ramp=True` default without exposing it, an inconsistency caught while reviewing the three scripts side by side.
5. **run_swiss_roll_svd_radar.py redesigned.** Previously: SVD init feeding `finsler_mds_joint.py::finsler_mds_freeB`, an Adam optimiser minimising the Finsler reconstruction loss $\sum (F(i \rightarrow j) - D_{\text{asym}}[i, j])^2$ – a fundamentally different mechanism (gradient descent against a target matrix) from the other two scripts' force-directed layout. Rewritten to use `randers_umap_fit` instead, per §3, so all three scripts are now directly comparable on the same update rule. `finsler_mds_joint.py` itself is untouched and still importable for anyone who wants the Adam/loss variant.

5 Analysis

5.1 Two distinct failure modes

- **Fragmentation** (all $n=300$ figures, all three methods, even with drift off): caused by the dense/full-batch repulsion in Eq. (2) having no stochastic negative sampling – clusters separate early and nothing pulls them back. Not fixed by anything in this report.
- **Central collapse** (Figure 1a, large n only): saturated $\|b_i\|$ nodes swing $\rho_{i \rightarrow j}$ by $\pm 100\%$ (Cauchy–Schwarz), breaking the (a, b) curve's calibration. Fixed by ramp (Figure 1b).
- These need different fixes – ramp solves collapse, not fragmentation.

5.2 Epoch budget is coupled to the ramp schedule

Figure 5: 300 epochs $\gg$ 100 epochs. Not just "more training" – B only reaches full strength ($s=1$) at $\tau/T \geq 0.70$ (Eq. (6)), so 300 epochs gives Y 210 epochs at full drift vs. only 30 at $T=100$. Epoch budget should be judged by post-ramp epoch count, not raw T .

5.3 k vs. locate-epochs: two different mechanisms

- **Why reducing k works.** The neighbourhood graph uses one global threshold, $\varepsilon(k) = \max_i d_k(i)$, where $d_k(i)$ is point i 's distance to its k -th nearest ambient neighbour. Since $d_k(i)$ is non-decreasing in k for every i , so is the max: $\varepsilon(k_1) \leq \varepsilon(k_2)$ for $k_1 < k_2$. On the

swiss roll, points on adjacent windings can be ambient-close (small Euclidean distance through empty space) yet geodesic-far (large true distance along the manifold, large $|t_i - t_j|$). If ε is large enough to connect such a pair directly, Dijkstra tunnels across windings – the classic Isomap short-circuit – and because ε is a single *global* value set by the sparsest point, it over-sizes the radius for every other point too, including exactly the points near winding boundaries where this happens. Reducing k shrinks $\varepsilon(k)$ monotonically everywhere at once, with no need to identify which points are at risk. Verified via geodesic- k NN t -jump (max/median $|t_i - t_j|$ among a point’s true geodesic k nearest neighbours): $k=20, n=300$ gives max 6.87 (75% of the full t -range – contaminated); $k=12, n=2000$ gives max 0.58, median 0.11 (clean).

- Locate-epochs↓ under-converges the locate step’s own embedding $\Rightarrow$ inflates $\|b_i\| = \|y_i^{\text{virt}} - y_i^{\text{real}}\|$ as a convergence artifact, not real drift.

5.4 Cross-method comparison

- Isomap vs. SVD-RADAR: same D_{asym} , same located B , only Y_{init} differs – cleanest controlled comparison. At matched 100-epoch budgets, neither shows a clear advantage yet; both still limited by fragmentation.
- isomap: no access to true ω , B comes from SVD of D_{asym} ’s skew-symmetric part – a structurally different signal, never checked against true ω . Treating it as comparable to the other two is currently an assumption, not a verified result.

5.5 Takeaways

1. Ramp scheduling should apply to *any* frozen B , not just located B – it fixes a property of Eq. (2), not of one construction method.
2. Epoch count and ramp schedule are coupled; comparing runs at equal T without accounting for post-ramp epochs risks misattributing a training-time effect (nearly happened with k above).
3. Fragmentation is still unsolved by anything in this report – only collapse was fixed.
4. Next: isolate k vs. locate-epochs (single-variable rerun); quantitative direction-accuracy of recovered B vs. true ω across all three methods.